

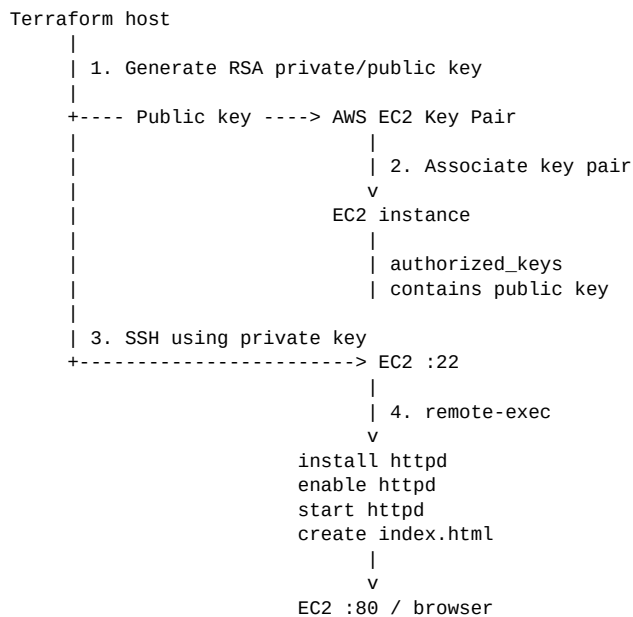
# Terraform remote-exec Demo

End-to-end AWS EC2 + SSH Key Pair + remote-exec + Apache HTTPD

## What this guide demonstrates

Terraform creates a completely new SSH key pair, registers the public key with AWS, creates an Amazon Linux 2023 EC2 instance, associates the key pair, connects to the instance over SSH, and uses remote-exec to install and start Apache HTTPD.

## Architecture



## Prerequisites

Before starting, have Terraform and the AWS CLI installed, have valid AWS credentials configured for the account, and use a region that supports the selected Amazon Linux 2023 AMI parameter. This example uses ap-south-1 (Mumbai).

## 1. Create the project

```
mkdir terraform-remote-exec-demo
cd terraform-remote-exec-demo
# Create a file named main.tf and paste the code below.
```

## 2. Complete end-to-end main.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  tls = {
    source = "hashicorp/tls"
    version = "~> 4.0"
  }
}
```

```

    }
}

# -----
# AWS Provider
# -----

provider "aws" {
    region = "ap-south-1"
}

# =====
# 1. GET THE DEFAULT VPC
# =====

data "aws_vpc" "default" {
    default = true
}

# =====
# 2. GET A SUBNET FROM THE DEFAULT VPC
# =====

data "aws_subnets" "default" {
    filter {
        name     = "vpc-id"
        values = [data.aws_vpc.default.id]
    }
}

# =====
# 3. GET THE LATEST AMAZON LINUX 2023 AMI
# =====

data "aws_ssm_parameter" "amazon_linux_2023" {
    name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

# =====
# 4. CREATE A BRAND-NEW SSH PRIVATE KEY
# =====

resource "tls_private_key" "demo_key" {

    algorithm = "RSA"

    rsa_bits = 4096
}

# =====
# 5. CREATE AWS EC2 KEY PAIR
# =====

resource "aws_key_pair" "demo_key" {

    key_name = "terraform-remote-exec-demo-key"

    public_key = tls_private_key.demo_key.public_key_openssh
}

# =====
# 6. SECURITY GROUP
# =====

resource "aws_security_group" "web_sg" {

    name          = "terraform-remote-exec-demo-sg"
    description   = "Security group for Terraform remote-exec demo"
    vpc_id        = data.aws_vpc.default.id

    # SSH
    ingress {

```

```

        description = "SSH"

        from_port = 22
        to_port   = 22

        protocol = "tcp"

        cidr_blocks = [
            "0.0.0.0/0"
        ]
    }

    # HTTP
    ingress {
        description = "HTTP"

        from_port = 80
        to_port   = 80

        protocol = "tcp"

        cidr_blocks = [
            "0.0.0.0/0"
        ]
    }

    # Outbound traffic
    egress {
        from_port = 0
        to_port   = 0

        protocol = "-1"

        cidr_blocks = [
            "0.0.0.0/0"
        ]
    }
}

# =====
# 7. EC2 INSTANCE
# =====

resource "aws_instance" "web" {

    ami = data.aws_ssm_parameter.amazon_linux_2023.value

    instance_type = "t3.micro"

    # Use the brand-new key pair created above
    key_name = aws_key_pair.demo_key.key_name

    # Use the first subnet from the default VPC
    subnet_id = data.aws_subnets.default.ids[0]

    # Attach security group
    vpc_security_group_ids = [
        aws_security_group.web_sg.id
    ]

    tags = {
        Name = "Terraform-Remote-Exec-Demo"
    }

    # =====
    # 8. REMOTE-EXEC
    # =====

    provisioner "remote-exec" {

        inline = [

            # Install Apache
            "sudo dnf install -y httpd",

```

```

# Enable Apache at boot
"sudo systemctl enable httpd",

# Start Apache
"sudo systemctl start httpd",

# Create a web page
"echo '<html><body><h1>Hello from Terraform remote-exec!</h1><p>Apache HTTPD was installed remotely.</p></body></html>'>"

# Verify Apache is running
"sudo systemctl status httpd --no-pager"
]

# =====
# SSH CONNECTION
# =====

connection {
  type = "ssh"

  # Amazon Linux 2023 default user
  user = "ec2-user"

  # Use the PRIVATE key generated by Terraform
  private_key = tls_private_key.demo_key.private_key_pem

  # Public IP of THIS EC2 instance
  host = self.public_ip

  # Give SSH enough time to become available
  timeout = "5m"
}
}

# =====
# 9. OUTPUTS
# =====

output "instance_id" {
  description = "EC2 instance ID"
  value = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP of the EC2 instance"
  value = aws_instance.web.public_ip
}

output "website_url" {
  description = "Apache website"
  value = "http://${aws_instance.web.public_ip}"
}

output "key_pair_name" {
  description = "AWS EC2 Key Pair created by Terraform"
  value = aws_key_pair.demo_key.key_name
}

```

### 3. Step 4 — Generate the brand-new key pair

```
resource "tls_private_key" "demo_key" {
  algorithm = "RSA"
  rsa_bits  = 4096
}
```

This resource does not create anything in AWS. Terraform generates an RSA public/private key pair during the Terraform run. The resource exposes both halves: **private\_key\_pem** and **public\_key\_openssh**.

```
tls_private_key.demo_key
|
+-- private_key_pem      -> private key used by Terraform for SSH
|
+-- public_key_openssh   -> public key sent to AWS
```

The private key is never sent to AWS as part of `aws_key_pair`. It is retained in Terraform's execution/state context and later supplied to the SSH connection.

### 4. Step 5 — Register the public key with AWS

```
resource "aws_key_pair" "demo_key" {
  key_name   = "terraform-remote-exec-demo-key"
  public_key = tls_private_key.demo_key.public_key_openssh
}
```

Terraform sends only the public key to AWS and creates an EC2 Key Pair object. The private key does not get uploaded to AWS.

### 5. How the key reaches the EC2 instance

```
tls_private_key.demo_key
|
| public_key_openssh
v
aws_key_pair.demo_key
|
| key_name
v
aws_instance.web
|
v
EC2 Linux user authorized_keys
|
v
PUBLIC KEY stored on instance

resource "aws_instance" "web" {
  ...
  key_name = aws_key_pair.demo_key.key_name
  ...
}
```

When AWS launches the Linux instance with that key pair, the corresponding public key is made available to the default SSH user's `authorized_keys`. This lets the matching private key authenticate.

### 6. How the SSH connection works

```
connection {
  type      = "ssh"
  user      = "ec2-user"
  private_key = tls_private_key.demo_key.private_key_pem
  host      = self.public_ip
  timeout   = "5m"
}
```

**type** selects SSH. **user** selects `ec2-user` for Amazon Linux 2023. **private\_key** supplies the private half generated in Step 4. **host = self.public\_ip** means the public IP of the current `aws_instance.web` resource — not the

Terraform host machine.

## 7. What happens during terraform apply

1. Terraform initializes the providers.
2. Terraform generates a new RSA key pair.
3. Terraform creates the AWS EC2 Key Pair using the PUBLIC key.
4. Terraform creates/uses the default VPC and a subnet.
5. Terraform creates the security group allowing TCP 22 and 80.
6. Terraform launches the EC2 instance and associates the key pair.
7. AWS assigns the EC2 a public IP.
8. `self.public_ip` resolves to that EC2 public IP.
9. Terraform establishes SSH using `ec2-user` + the PRIVATE key.
10. `remote-exec` runs inside the EC2 instance.
11. `httpd` is installed, enabled, started, and given an `index.html`.
12. Terraform prints the instance IP and website URL.

## 8. Run the demo

```
terraform init
terraform plan
terraform apply

# Type:
yes

# After completion:
terraform output public_ip
terraform output website_url
```

Open the resulting website URL in a browser.

## 9. Important security notes

This is a learning/demo configuration. The SSH rule allows 0.0.0.0/0 so the connection is easy to demonstrate. In production, restrict SSH to trusted IPs or use a private access mechanism such as AWS Systems Manager Session Manager. Also note that `tls_private_key.private_key_pem` becomes part of Terraform state, so protect the state file and its backend.

## 10. Cleanup

```
terraform destroy
```

Confirm the destroy operation when prompted. This removes the EC2 instance, security group, AWS key pair, and Terraform-managed resources created by the demo.

End of guide